

Rationale

In the rapidly evolving field of computational biology, selecting appropriate analytical methods is crucial for deriving meaningful insights from increasingly complex datasets. Throughout our journal club sessions this semester, we have systematically reviewed cutting-edge research applying machine learning approaches to biological data analysis.

These discussions revealed notable patterns in methodological choices that warrant further exploration and synthesis.

Our class has traditionally focused on single nucleotide polymorphism (SNP) data, analyzing genetic variations at the most granular level to understand their associations with phenotypic traits and disease susceptibility. SNP-level analyses have proven valuable for identifying specific genetic markers and have been foundational to genome-wide association studies (GWAS). However, they often present significant computational challenges, particularly when dealing with high-dimensional data that includes millions of variants across thousands of samples.

In contrast, the paper we reviewed for this project adopted a gene-centric analytical approach, effectively "zooming out" from individual nucleotides to examine variation and expression at the gene level. This shift in granularity, while sacrificing some fine-detail resolution, offers several compelling advantages. Gene-level analyses can potentially reduce dimensionality, mitigate multiple testing burdens, improve interpretability through direct biological context, and enhance statistical power by aggregating signals across multiple variants.

Data Context

The data we use for this project provides information on 12534 genes (number of columns) belonging to 174 observations (number of rows). Below is a snippet of what it looks like.

Class	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	D11	D12	D13	D14	D15
7	153	228	32	30	-36	48	984	-19	537	1886	3402	131	-51	202	2
7	154	99	43	55	66	63	5051	-26	1001	1315	2309	37	-77	167	3
7	84	85	19	28	-104	28	2387	-80	1131	2457	2290	51	-63	157	-16
7	234	169	40	36	81	6	2657	-6	1214	1944	2313	-10	-28	338	32
7	104	58	42	13	107	5	3562	18	1464	1385	3127	77	-37	209	-30
7	207	408	56	6	41	33	2617	40	1243	1142	2223	6	-89	187	0
7	188	245	45	18	-15	44	1885	25	1313	1521	2029	36	-40	284	-11
7	221	132	84	19	81	74	2589	-21	742	1892	2409	6	-81	684	-11
7	198	328	13	41	16	38	2536	-35	1006	1307	2921	100	14	191	3
7	303	261	53	71	-40	66	1184	-28	711	2672	1203	29	-97	233	-9
1	247	348	65	89	28	6	27	-54	1857	1185	1656	0	-74	411	-11
1	173	380	54	87	-11	-27	750	-9	1458	985	878	55	-11	322	2

The columns that start with capital letter D represent the genes and the `Class` column consists of 11 tumor types we would like to identify. The values under any of the D columns indicate how `on` a gene is during protein production. In other words, the larger the value, the more actively that gene participates in such process.

Background Information

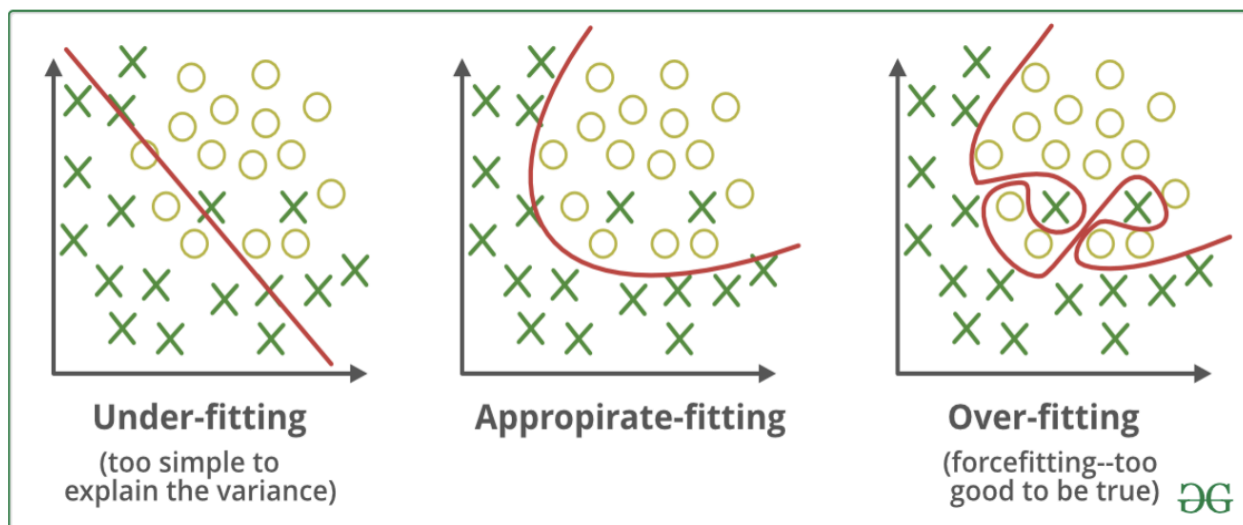
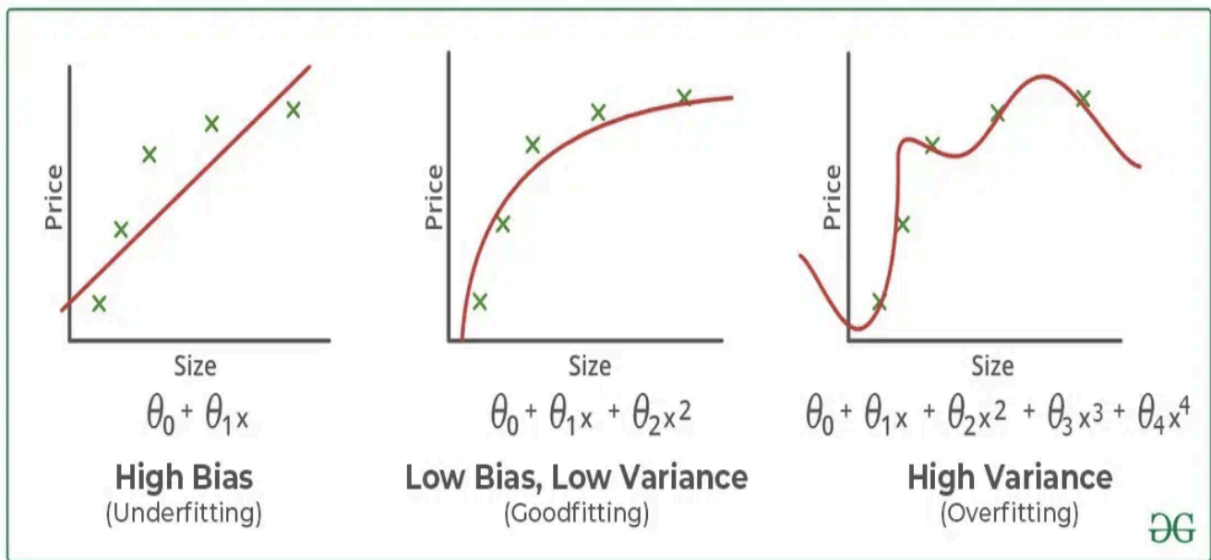
Machine Learning

Defined most broadly, machine learning is a field concerned with the development and application of statistical algorithms that can learn from seen and then generalize onto unseen data, performing tasks and quantifying relationships without explicit instructions. Machine learning is widely used across almost every industry in the world as a robust tool for prediction across regression and classification tasks. Rather than simply fitting a regression like we might in a class like Stat 155, a machine learning approach has us split the data into `train` and `test` sets. We use the training set to calibrate and `tune` our model, then at the very end use the unseen set aside test data as the true measure of performance, simulating the application of the model in the real world.

One of the most common challenges faced in machine learning is the 'curse of dimensionality', which arises when the efficiency and effectiveness of an algorithm or model deteriorates as the dimensionality of the data increases, leading to overfitting, spurious correlations, and computational complexity. This will be a recurring problem our models seek to address in this project.

Overfitting is an abstract but critical term to understand when building or interpreting machine learning models, and it applies to both regression and classification prediction tasks. The 'fit' of a model, or how the model understands the relationship or shape present in the data, can be roughly categorized in three ways:

1. **Underfit:** A model is underfit when it is too simple to capture the true relationship in the data. As we see in both of the visual examples below, an underfit model might ascribe a linear relationship to two variables, when the relationship is more complex. Since the model is failing to capture the true complexity of the relationship, it is underfit.
2. **Overfit:** The opposite of underfit, but also bad! A model is overfit when it mistakes random noise or idiosyncrasies in the training data for generalizable aspects of the true relationship between the variables. The model clings too closely to the training data, creating a force or too good to be true fit that fails to generalize to new data with different random noise.
3. **Well-Fit.** The goal. A well fit model strikes the balance between under and overfitting, capturing the true relationship present in the data while not mistaking random noise for true trends. While a well-fit model won't be initially perfect on the training set like an overfit model, it will perform better on unseen data (which is where performance matters).



However, what happens when we can't visually verify the fit of our model like we can in the 2-d figures? In the context of our project (and many others) high dimensional data and models make such easy and intuitive observation difficult. Instead, we'll use comparisons between the training and test performance as our metric to avoid overfitting. If our test accuracy is meaningfully lower than our training accuracy, it's good evidence that our model is over-adjusting to the training data at the cost of generalizability.

Models

LASSO

LASSO is a machine learning algorithm that stands for Least Absolute Shrinkage and Selection Operator. It is an ordinary least squares regression with a penalty term called lambda. The goal is to minimize the entire function.

$$\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

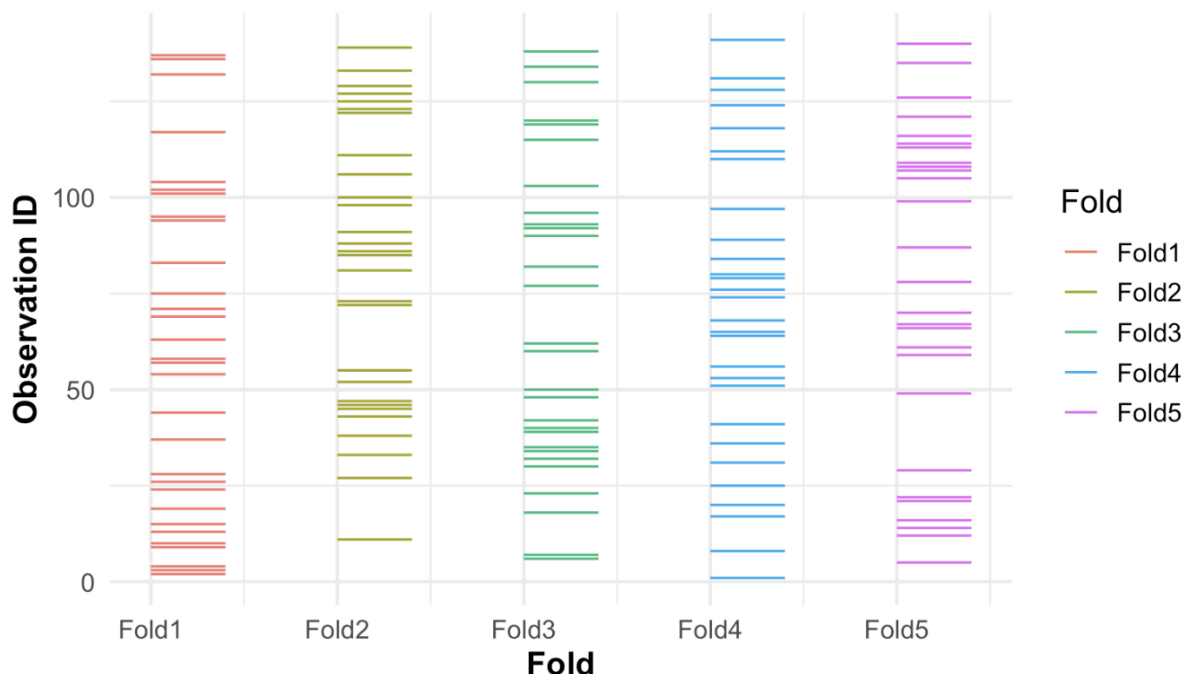
This works by shrinking the coefficients, beta, down towards zero when they don't add much to the regression. This is useful in the context of genetic data because most large genetic datasets have more columns than observations, so ordinary least squares cannot be run. If coefficients are shrunk to zero that effectively eliminates columns in the data so we are left with more observations than variables. In the coming paragraphs we will walk



through how to run LASSO in r and what some of the lines mean, using our data. This can be followed along using the qmd file in the same folder as this README. This framework can be applied to any form of data, not just the data we used. We use the test and training splits that were used earlier, so that our metrics are being compared accurately.

We pull that data in and make all genes numeric and the Class variable a factor for both the test and training data. Then we split the data into five folds for cross validation.

5-Fold Cross-Validation: Test Observations per Fold

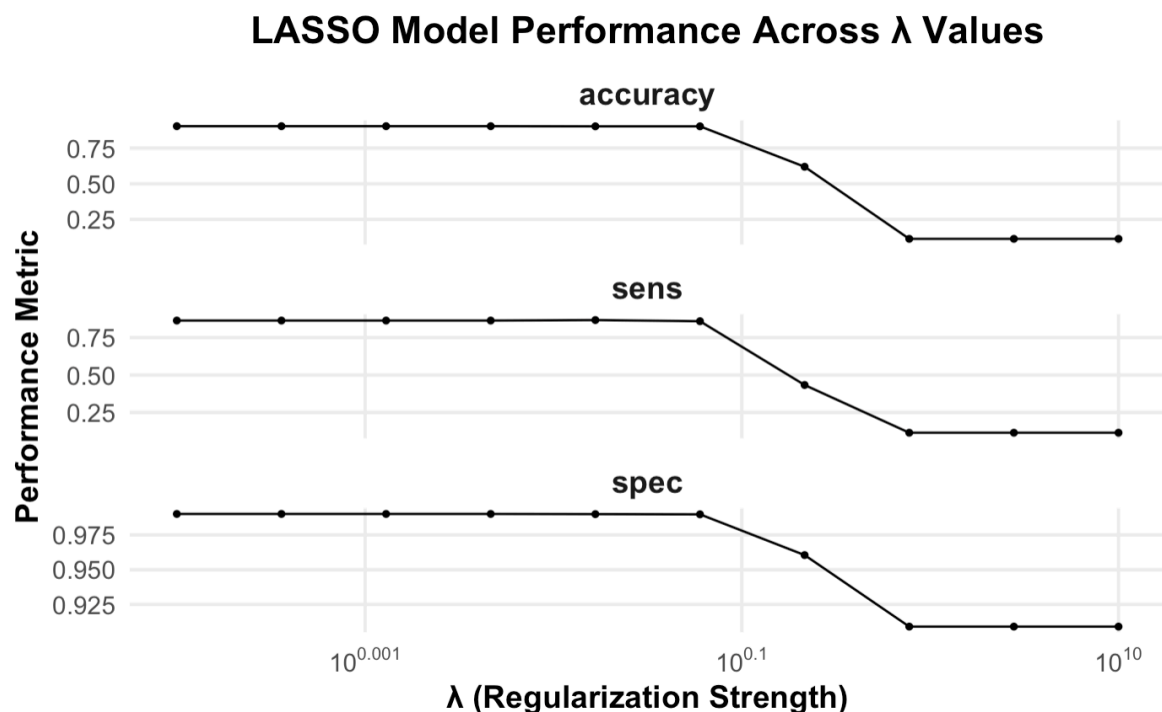


After that we define the model specification for the multinomial LASSO regression. This code chunk works by using the `multinom_reg()` function from the `parsnip` package, with `penalty = tune()` indicating that the regularization strength will be tuned later during model selection. The `mixture = 1` parameter sets the regularization to be pure LASSO, rather than Ridge Regression or a mix of both. The model is then assigned the `glmnet` engine, which fits regularized generalized linear models. Lastly, `set_mode("classification")` tells `parsnip` that this is a classification model.

The `lasso_rec` creates a preprocessing recipe for the modeling workflow. The formula `Class ~ .` specifies that the outcome variable is `Class`, and all other variables in the dataset `tumor_train` will be used as predictors. Then it standardizes all numeric predictor variables by centering them to have a mean of zero and scaling them to have a standard deviation of one. This normalization is important for models like LASSO, which are sensitive to the scale of the input features. The resulting `lasso_rec` object defines how the data should be preprocessed before being passed to the model during training and evaluation. Then the workflow combines the recipe and model so that the data is always prepared the same way before being used by the model.

We create a grid of ten lambda values to test how different levels of regularization affect the model's performance. After that we perform hyperparameter tuning by testing different values of the regularization parameter (lambda) to find the best-performing version of my LASSO model.

`tune_grid(...)`: Tries out a bunch of different lambda values from `penalty_grid`, one by one, to see which one helps the model perform best. It does that using the workflow previously defined and the metrics line says what metrics to look at, while the control line saves the results. We then graph these results and put them in a table. The next important line of code is selecting the best lambda, which is done with the `select` by one standard error function, which chooses the best lambda based on accuracy. It does this by using the tune output data that was previously created, putting them in descending order of accuracy, and picking the best lambda within one standard deviation of the actual best lambda. We also ran this for sensitivity and found that the penalty term was the same, but this may not be the case if this were to be run on other data. Below is a graph of how each metric changes as lambda increases. As lambda increases all three metrics stay the same and once lambda gets large enough they start to drop off. Showing how as too many coefficients get shrunk to zero, the model loses accuracy.

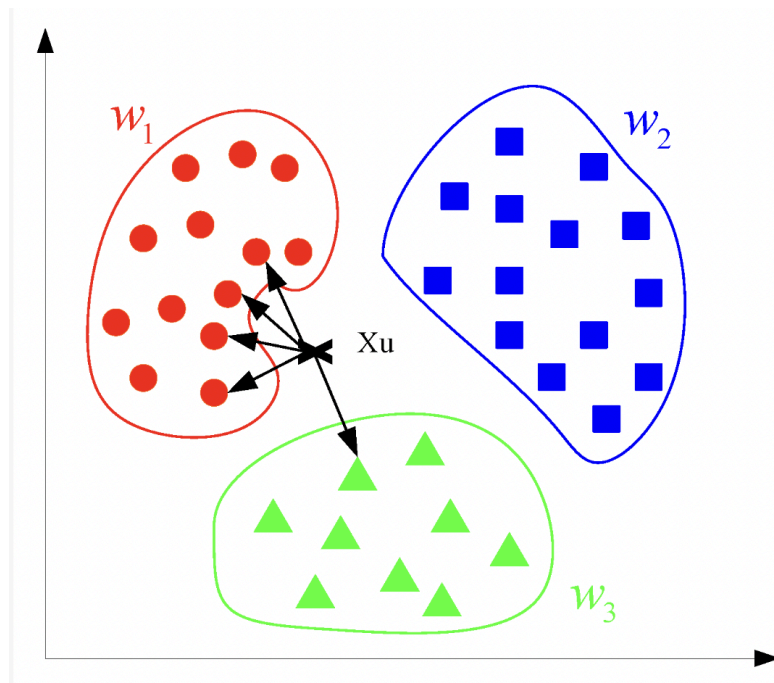


Next we fit the data using finalize workflow, which makes sure to use the best lambda on the workflow previously created then fits the data to all of the training data. We then make predictions on the test data set. The code bind_cols places each prediction alongside the actual class label and all the original features from the test dataset. We also do this for the best lambda by sensitivity as a sanity check, to make sure the numbers are the same.

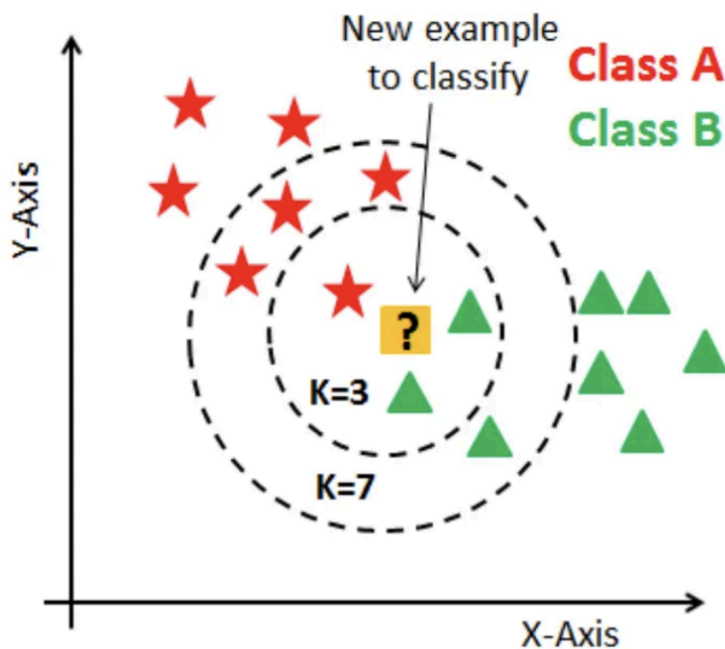
We then take the final fit data and filter out all the coefficients that were not shrunk down to zero and do the same from the sensitivity code. In this case we were left with 111 predictors, cut down from the thousands that were previously in the data. We then make a confusion matrix showing how accurate our predictions were. The rest of the code is outputting the metrics into a data table.

KNN

K-nearest neighbors, or KNN for short, is a classic but powerful machine learning algorithm that classifies a data point on the basis of its k nearest neighbors. After a query point is selected, a majority vote of the nearest neighbors is used to assign the new point to the data class which has the most representatives within the nearest neighbors of the point.

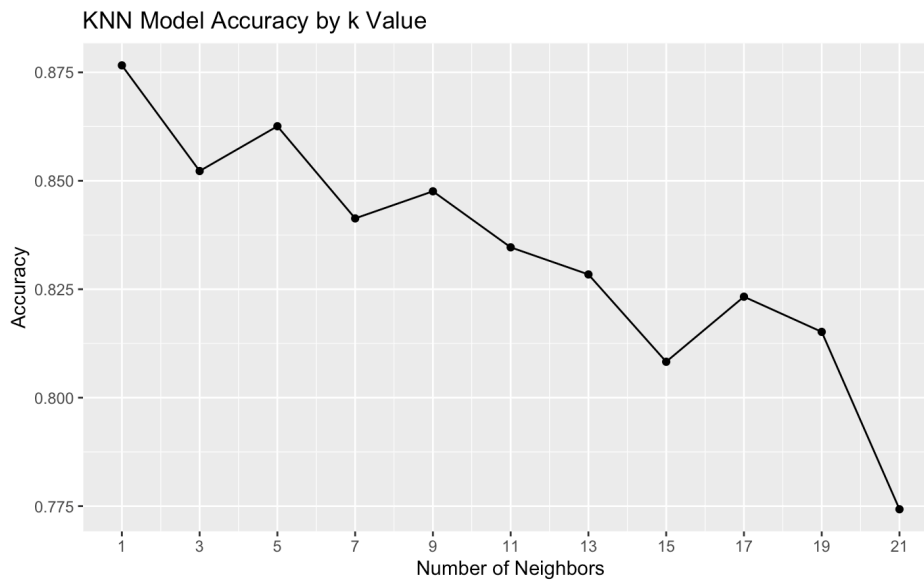


KNN is a powerful algorithm because it is non-parametric, in other words it doesn't make assumptions about the underlying data. While a linear regression model inherently assumes a linear relationship, a model like KNN doesn't, allowing it to capture nuanced classification and decision boundaries of complex relationships. KNN is also powerful because of its simplicity, including only a few hyperparameters, or adjustable elements of the model that can be used to refine performance. While 'k', or the number of neighbors polled to classify a new point, is the main hyperparameter, one can also experiment with the type of distance calculation used to define the 'nearest' neighbors in the first place (we use classic Euclidean distance).

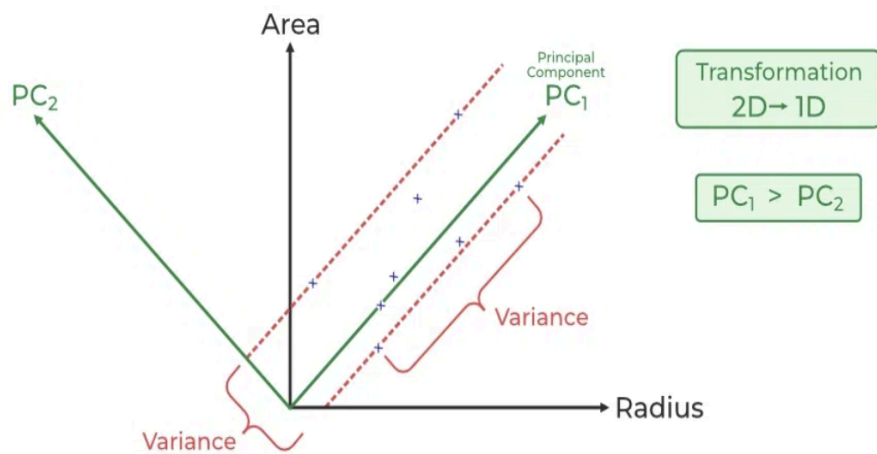


The behavior of the model changes as we adjust the number of nearest neighbors we take into account, demonstrated by the figure above. If we only use 3 neighbors, we'll classify the query point as a triangle. However, if we use 7, we'll classify it as a star. In general, the fewer neighbors we use, the more volatile and dynamic our model becomes (adjectives with positive connotation, but not inherently best in this context!), increasing the variance but decreasing the bias. As we use more neighbors, the model becomes slower to adjust, decreasing variance but increasing bias.

Shown by the plot below, after testing the performance of a variety of k values, using a single nearest neighbor yielded the best accuracy for our KNN model.



This model used two strategies to address the curse of dimensionality, or $p > n$ problem, eventually yielding two distinct KNN models. The first was aggressive feature selection, narrowing down the genes to only the most 100 predictors. The second was the use of Principal Component Analysis, or PCA, as a feature selection tool. PCA attempts to reduce the dimensionality of the data while preserving as much information as possible by decomposing the predictor variables into linearly independent or orthogonal principal components that are then used by the model as predictors. We'll see how these strategies panned out in the results section.

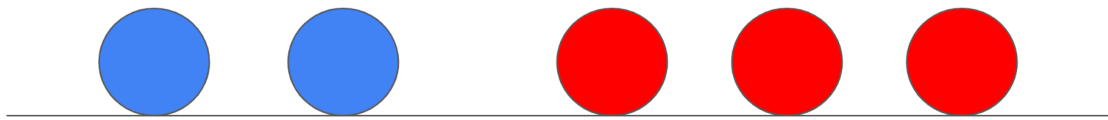


We took a couple of other standard machine learning steps when fitting this model, using 10-fold cross validation and normalizing feature scales to ensure no single gene dominated on the basis of a large numerical range, which ensured the validity of our distance measure.

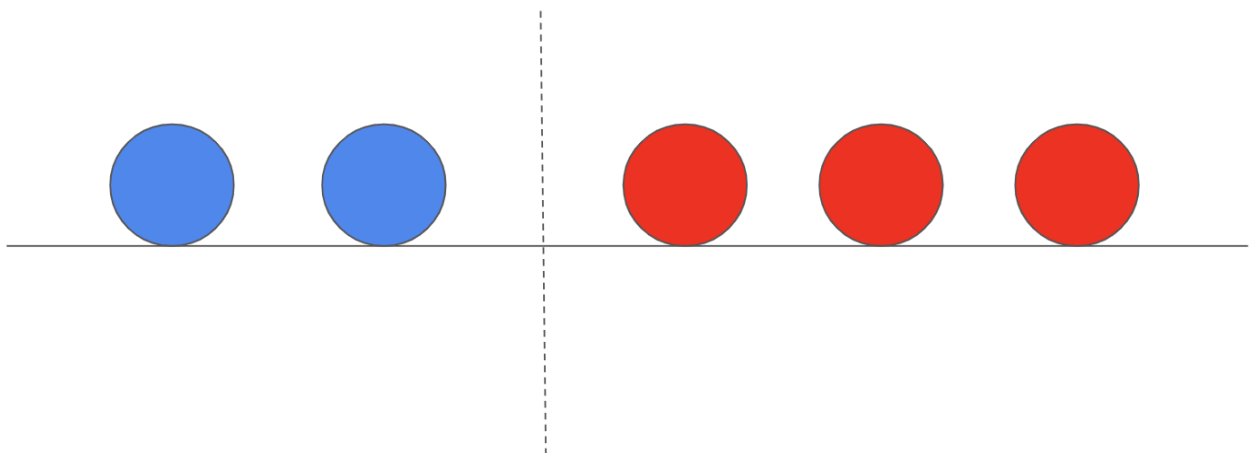
SVM

How it works

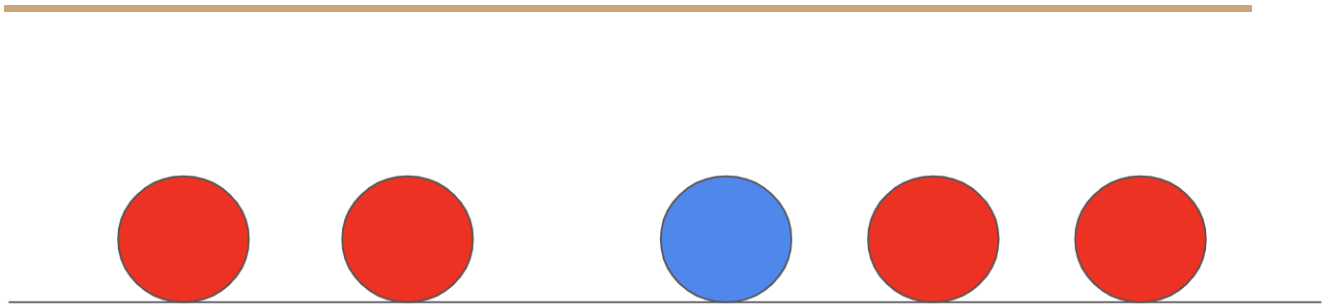
The core of support vector machines lie in their power to increase the dimensional space our data currently lives in. To illustrate why this is important, here is a little brain teaser activity. Given a set of data points, can we find a line that separates the blue circles from the red ones?



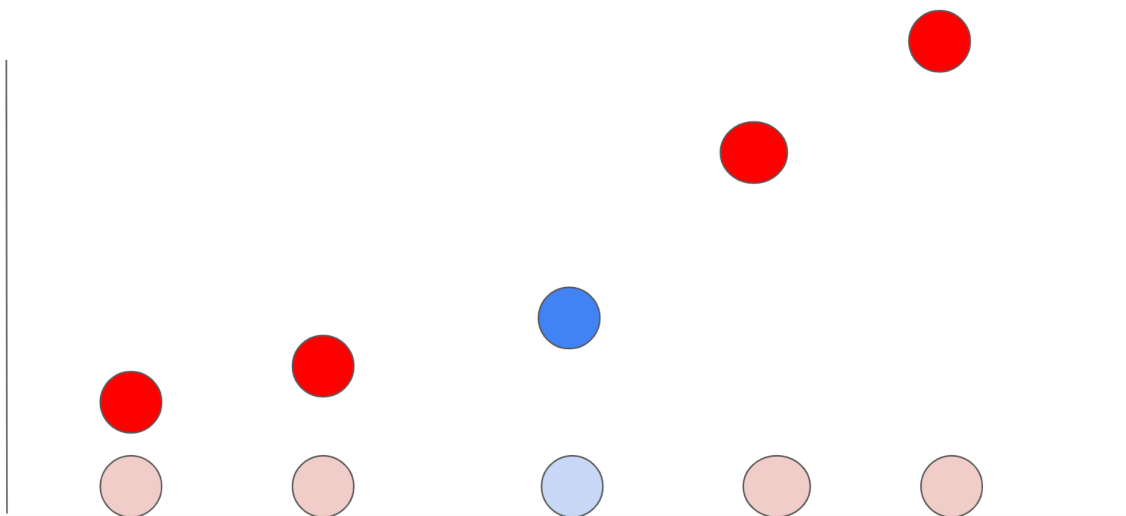
In this particular case, it is rather straightforward to pick as line that groups those circles by the color, as is shown below:



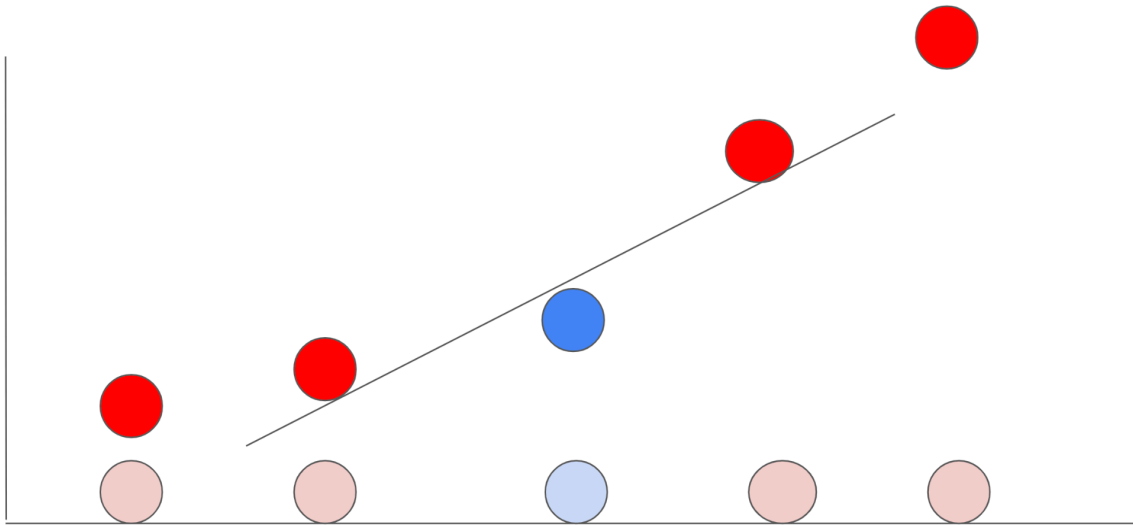
However, real life data tends to be more complicated than the example provided above. In fact, let's consider this next case:



In this scenario, the answer might not be as obvious. As one soon may find out, there does not exist a line such that the red and the blue circles are clearly separated. To solve this problem, one useful trick is to change how the data points are represented by increasing their current dimensional space. Specifically, one can choose to square the values of the circles on the x axis (now a 1-dimensional space), effectively converting them to data belonging to a new 2-dimensional space.



After the transformation, the task of finding the best decision boundary becomes slightly easier than before. For this very simplified example, such dividing line can be constructed as below



In actuality however, the algorithm does not physically change the data. It only does so “abstractly” purely for the purpose of finding the best boundary. Our data therefore remains intact.

Kernel

The conversion we demonstrated above can be dictated by a hyperparameter required for SVM models, called the “kernel”. Some of the most commonly used ones include linear, polynomial and radial. The process of squaring data in the previous example is done through the “polynomial” kernel. As for the radial kernel, it is particularly useful in dealing with highly non-linear data and to understand how it works is no easy task, Radial type is a special, more complex case of transformation where data is augmented to some infinite dimensional space, so intricate that there is no way to visually illustrate it.

It is also worth mentioning that as we play with these types of kernels and compare how the model performs accordingly, we may derive some useful insights from the linearity of the data we are using. For example, after running 3 aforementioned kernel types on the tumors dataset, we learned that accuracy decreased the more complex we adjusted our model to be

Overall Statistics

Accuracy : 0.8491
95% CI : (0.7241, 0.9325)
No Information Rate : 0.2075
P-Value [Acc > NIR] : < 2.2e-16



Linear

Overall Statistics

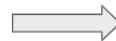
Accuracy : 0.3774
95% CI : (0.2479, 0.5211)
No Information Rate : 0.2075
P-Value [Acc > NIR] : 0.003403



Polynomial

Overall Statistics

Accuracy : 0.0943
95% CI : (0.0313, 0.2066)
No Information Rate : 0.2075
P-Value [Acc > NIR] : 0.9916



Radial

We can see from the image above that the linear version outperformed the other two, with the radial classifier underperforming relative to the no-information rate, meaning it performs worse than guessing the most frequent class.

So what can we say about the data? It is linear enough that we only need a linear transformation to complete the classification task successfully, without the need to use more complicated kernels, which may end up exacerbating the results.

Results/Conclusions

Overall Accuracy

Overall Accuracy vs. Model



After training all models and comparing the predictions against the test set, the accuracy results can be interpreted as follows:

SVM achieved the highest overall accuracy, with LASSO and KNN performing comparably well, whereas KNN with PCA lagged considerably behind the other models.

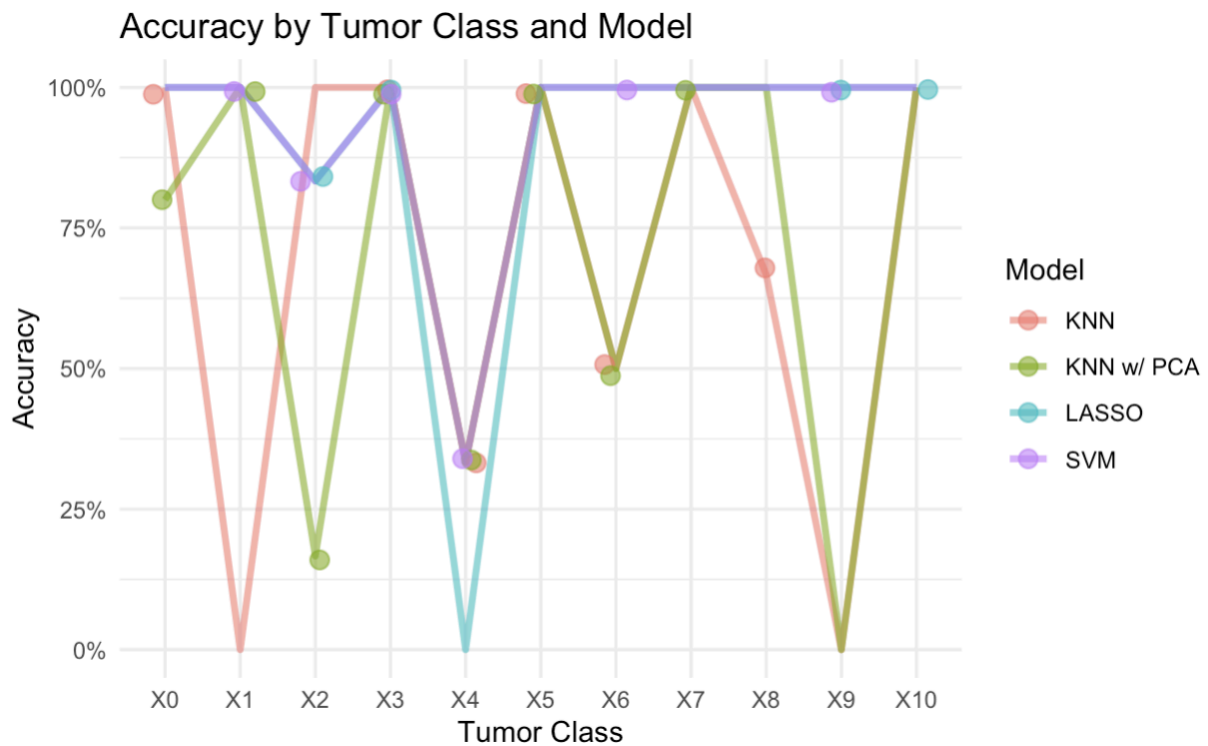
The results for KNN and KNN with PCA contradicted our expectations that incorporating PCA would improve KNN's performance. However, the opposite occurred, likely due to the nature of PCA. PCA reduces dimensionality by selecting components that maximize variance, without regard to class label relevance. As a result, it may have discarded features that were important for distinguishing between classes, leading to an underfitting model. This distortion of the feature space made distance-based comparisons—crucial for KNN—less effective. In contrast, feature selection via Random Forest preserved

class-relevant information, enabling KNN to better capture the structure necessary for accurate classification.

Accuracy by Class

The second results metric we used was model accuracy by class. The conclusions are interesting. In general, every model struggles on Class 4. The KNN models struggle in different ways, with the best features model uniquely struggling on Classes 1 and 8, and PCA doing worse on class 2. SVM seems like the best overall, maintaining a high accuracy percentage across all classes. Even when it does poorly, like on Class 4, it's no worse than the other models. LASSO is on average more consistent than KNN, but is the worst on Class 4.

Accuracy by class is important because it gives us a more granular understanding of the overall accuracy score, and means that if we were to use a team or ensemble of models to make a prediction, we could use accuracy by class as a tiebreaker if two models disagreed, but one had superior accuracy in a given class area. However, it's important to keep in mind the limitations of this type of plot when working with such a small dataset. Using an 80/20 train/test split, our models were only exposed to 33 test data points, or just over 3 prediction attempts per class. Unfortunately, this makes it difficult to say for certain how our models would do on a given class absent more observations to establish a more robust trend.



Sensitivity/Specificity

The final metrics we used were sensitivity and specificity. Sensitivity is the true positive rate and specificity is the true negative rate. When predicting a certain class, if we were to predict not that class our specificity would still be very high, which is why we care the most about sensitivity. We want to find, of the actual positives, how many did we actually predict to be positive. We find that KNN and LASSO have nearly identical sensitivities and SVM is close behind. KNN with PCA is lower but still has a high enough sensitivity to use if necessary. We wouldn't recommend using a single metric rather a combination of these, with the first two choices being KNN and LASSO.

Model <chr>	Sensitivity <dbl>	Specificity <dbl>
KNN	0.8958333	0.9919165
SVM	0.8750000	0.9878676
LASSO	0.8939394	0.9877675
KNN (PCA)	0.7962963	0.9773057

Main Conclusion

The three models—SVM, LASSO, and KNN (without PCA)—achieved similarly high overall accuracies. Among them, SVM demonstrated the best performance in terms of both overall and per-class accuracy. While KNN and LASSO occasionally outperformed SVM in metrics such as sensitivity and specificity, the differences were minimal, typically within a margin of 0.01 to 0.02.